07

백트래킹

선택을 시험하고, 돌아와서 다른 선택을 살펴보는 탐색

선택을 차례로 시험하는 탐색

길 찾기 막다른 길을 만나면 마지막 갈림길로 돌아옵니다.
거기서 아직 가보지 않은 길을 고릅니다.

문제 풀이 어느 선택이 답이 될지 모르면 하나씩 시험합니다.
하나를 고른 뒤, 그 선택에 이어 다음 것을 고릅니다.

돌아가기 끝까지 확인했거나 더 진행할 수 없으면
방금 선택을 취소하고 다른 것을 골라봅니다.

백트래킹 **앞으로 진행했다가 이전 선택으로 돌아오면서
가능한 답을 찾는 탐색 방법입니다.**

되돌아가기와 가지치기

후보 후보는 지금 고를 수 있는 값입니다.
a·b·c 중 고른다면 후보는 세 개입니다.

끝까지 탐색 가능한 답을 하나 만들었으면 저장하거나 셉니다.
다른 답을 찾기 위해 앞의 선택으로 돌아갑니다.

가지치기 계속 해도 답이 될 수 없다는 사실을 알았다면
그 선택 뒤의 경우들은 더 확인하지 않습니다.

진행 순서 **먼저 문자 예제로 선택과 되돌리기를 살펴보고,
그다음 합·색칠·퀸 문제에서 조건을 검사합니다.**

전화 문자 조합 문제

상황

숫자 키에 연결된 알파벳을 골라
새 문자열을 만듭니다.

주어진 값

이번 예제의 입력은 "25"입니다.
2에는 a·b·c, 5에는 j·k·l이 연결되어 있습니다.

규칙

첫 글자는 a·b·c 중 하나, 둘째는 j·k·l 중 하나.
숫자의 순서를 바꾸거나 건너뛰지 않습니다.

구할 답

aj, ak, al, bj, bk, bl, cj, ck, cl
가능한 아홉 문자열을 모두 담은 목록입니다.

숫자마다 고를 수 있는 문자가 있습니다

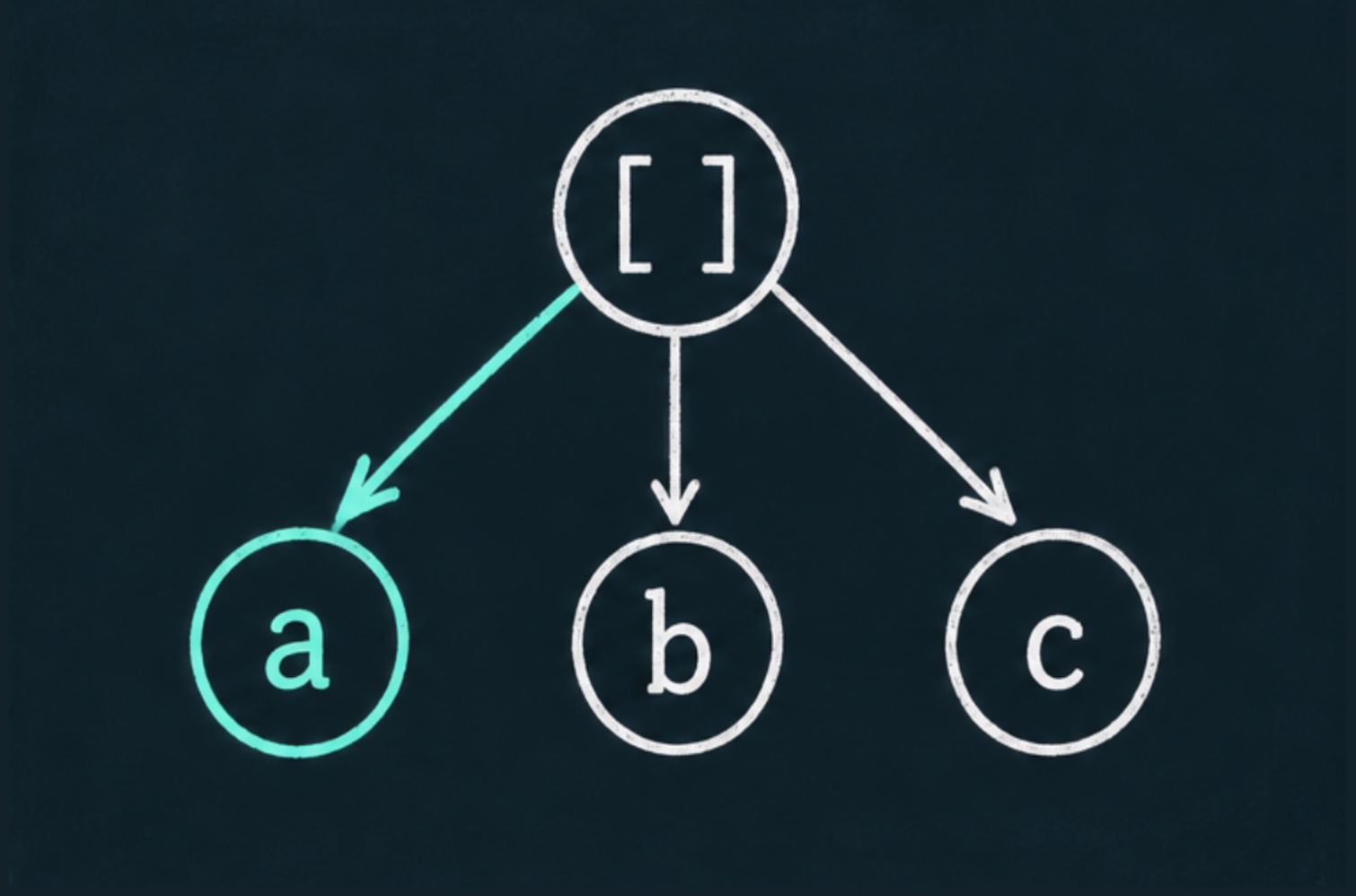
2에서는 a·b·c, 5에서는 j·k·l

입력 "25"는 두 자리입니다.

각 숫자에서 문자 하나씩 고릅니다.

만들 문자열의 길이는 2

첫 자리에서 갈래가 세 개 생깁니다



아직 아무 문자도 고르지 않았습니다

첫 숫자 2의 후보는 a, b, c입니다.

a를 고른 갈래부터 끝까지 봅니다.

빈 선택 []에서 출발

a를 골랐으면 첫 자리는 그대로 둡니다

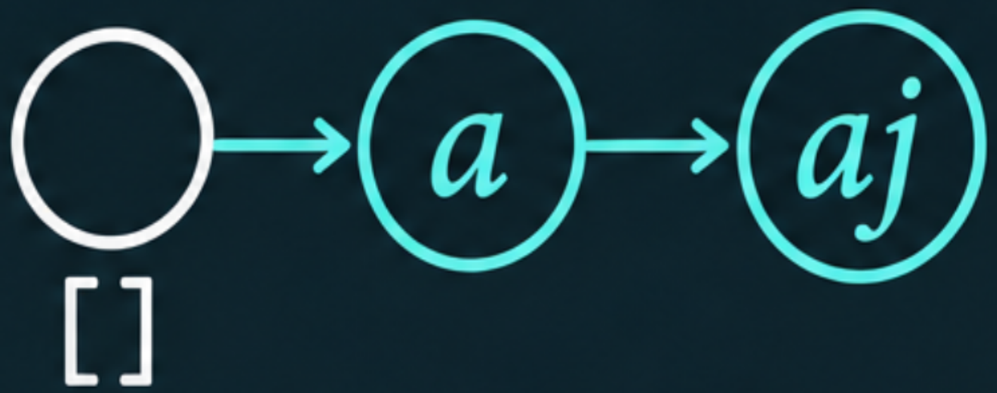
현재 선택은 [a]입니다

첫 자리의 선택이 끝났습니다.

이제 둘째 숫자 5를 봅니다.

다음 후보는 j, k, l

j를 붙이면 aj가 완성됩니다



입력 두 자리를 모두 처리했습니다

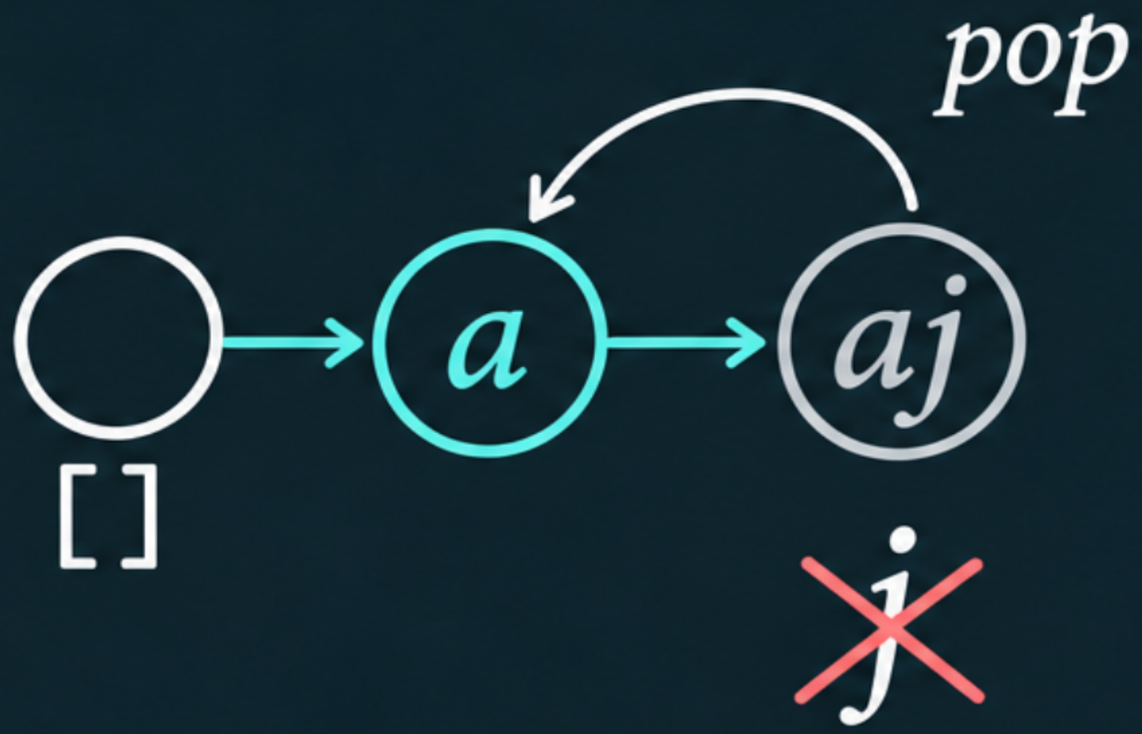
현재 선택은 [a, j]입니다.

완성한 "aj"를 답 목록에 저장합니다.

aj

답 목록에 "aj" 추가

j만 지우고 a로 돌아갑니다



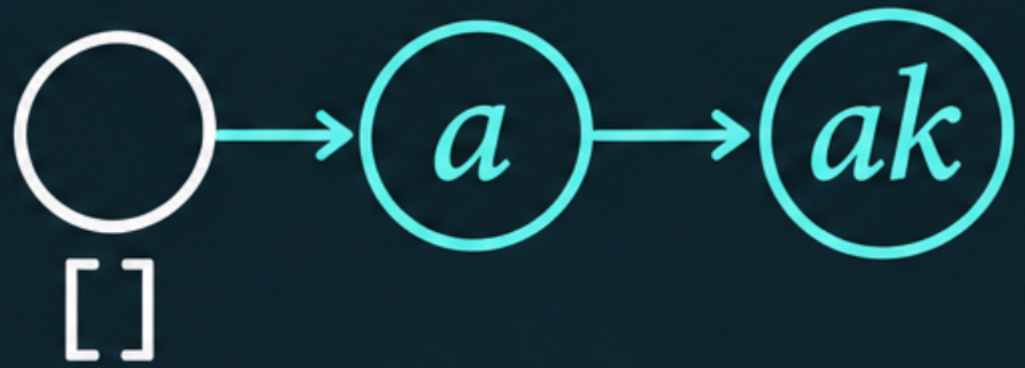
방금 고른 마지막 문자만 취소

저장한 답 "aj"는 남겨둡니다.

현재 선택은 다시 [a]가 됩니다.

이제 k를 고를 수 있습니다.

같은 a 뒤에 이번에는 k를 붙입니다



aj ak

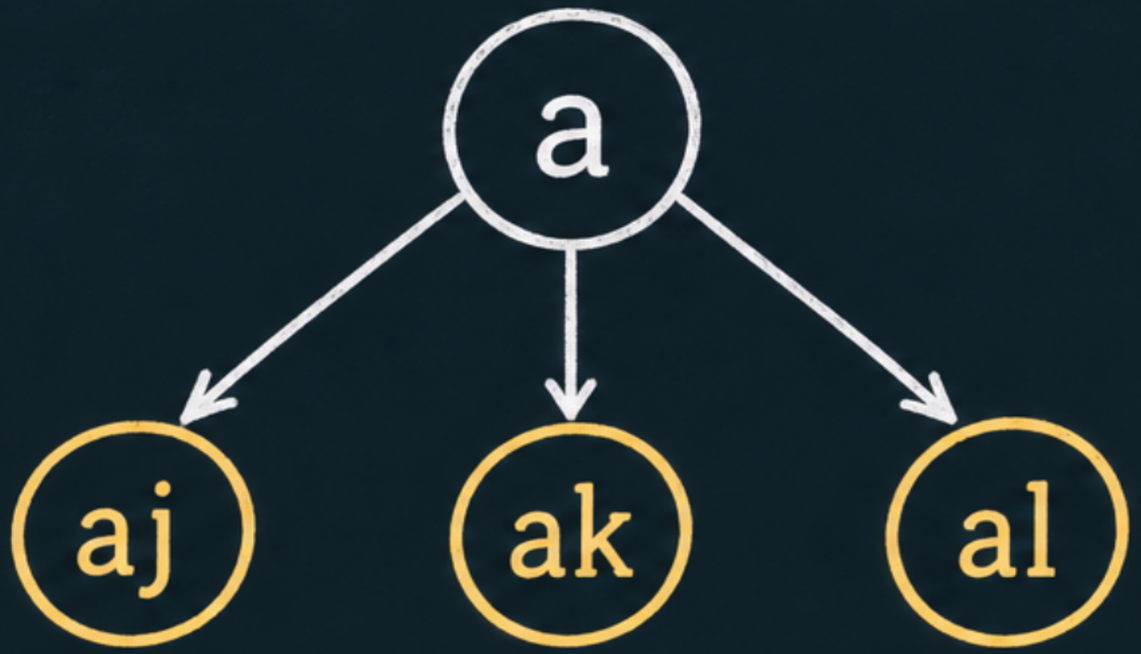
돌아온 자리에서 다음 후보를 선택

현재 선택 [a]에 k를 붙입니다.

"ak"를 답 목록에 추가합니다.

aj, ak, 그리고 al

a 아래의 세 답을 모두 찾았습니다



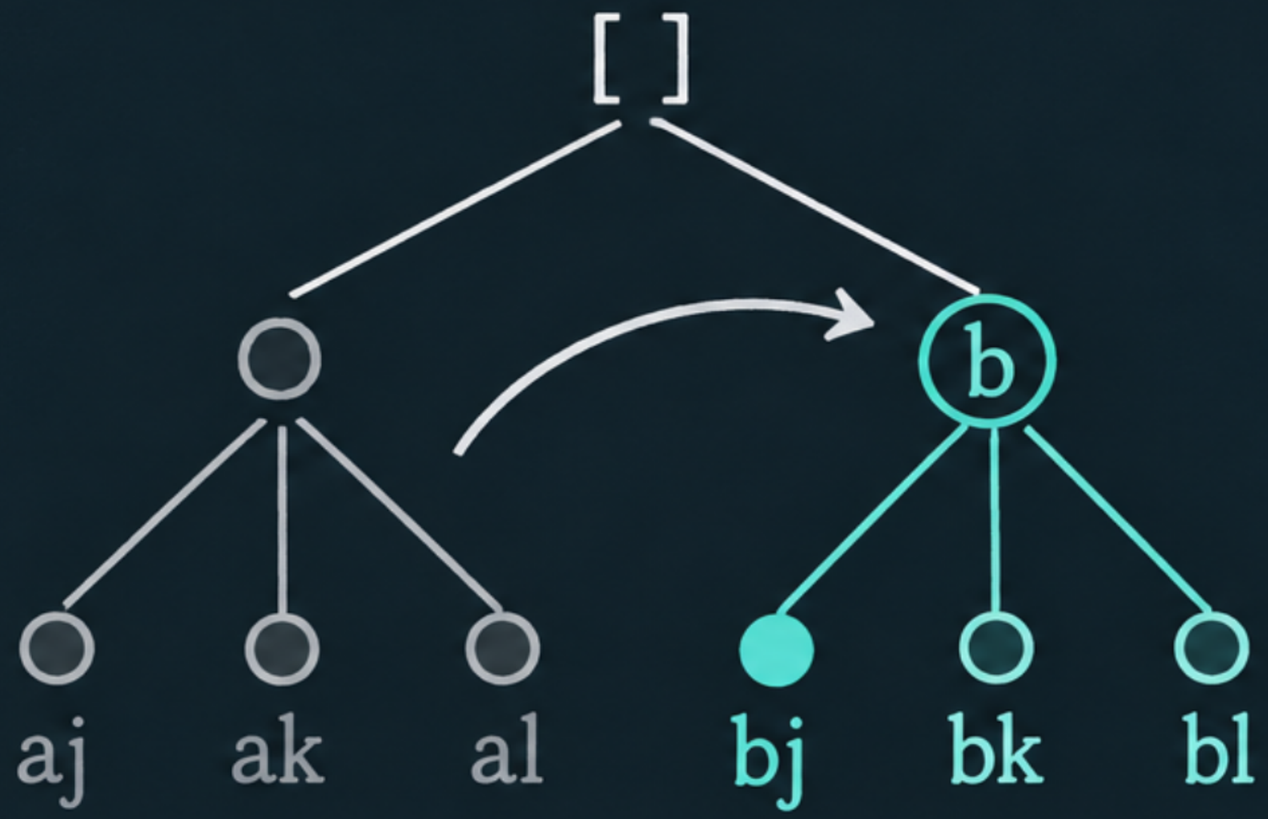
둘째 자리의 후보 j·k·l을 모두 확인

aj, ak, al을 저장했습니다.

이제 첫 자리 a도 취소합니다.

첫 자리의 다음 후보는 b

첫 자리도 돌아와서 바꿉니다



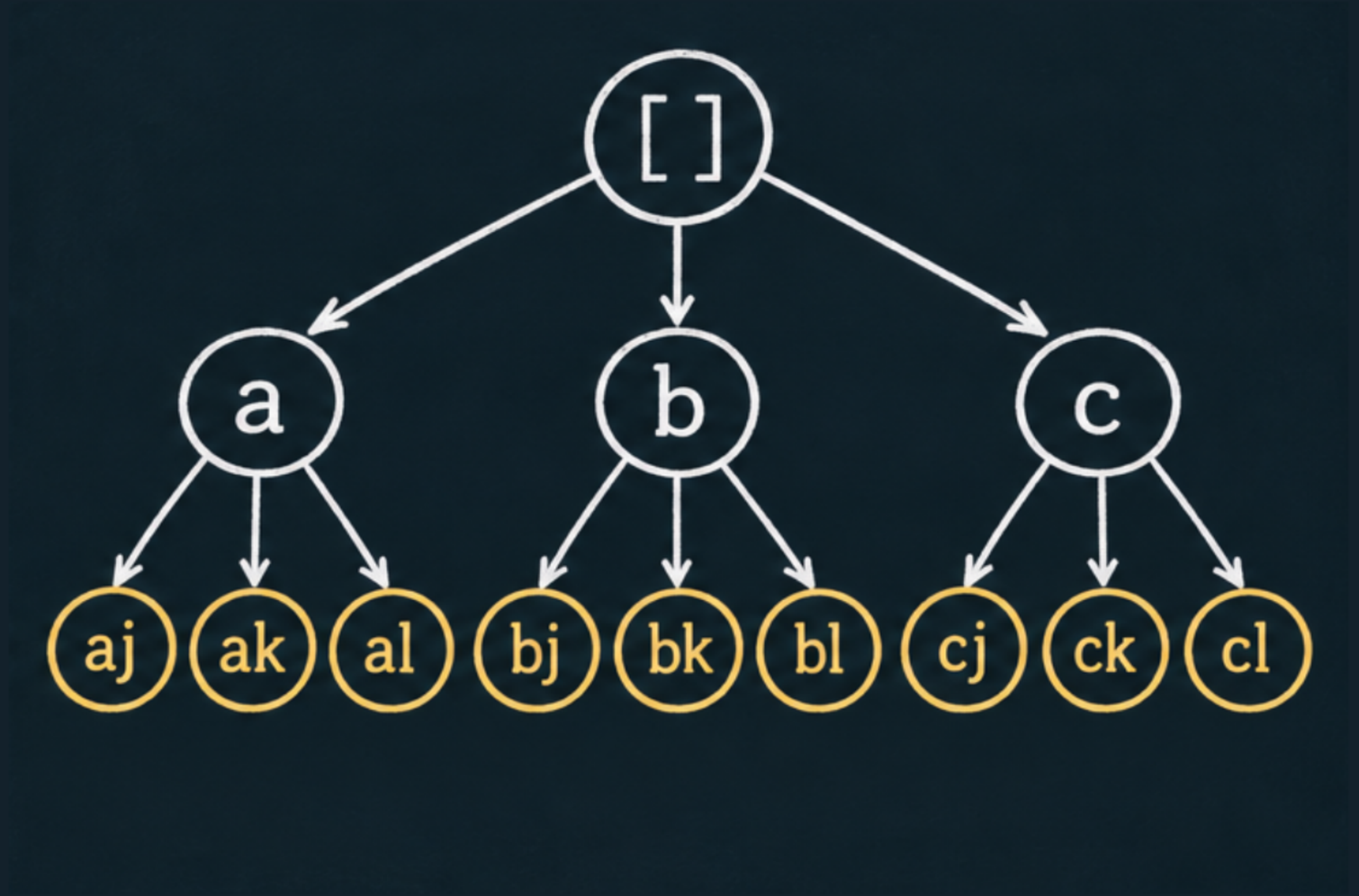
a 갈래를 마치고 b 갈래로 이동

b를 고른 뒤 j부터 다시 시작합니다.

그다음 c 갈래도 같은 방법입니다.

어느 갈래도 빠뜨리지 않기

25에서 만드는 아홉 개의 답



끝에 도착한 갈래 하나가 답 하나

a 아래 3개, b 아래 3개, c 아래 3개

3 × 3 = 9개의 문자열

aj ak al · bj bk bl · cj ck cl

작업 중인 글자와 완성한 답은 따로 둡니다

digits 입력 "25"는 바뀌지 않습니다.
어느 숫자까지 처리했는지만 이동합니다.

path 지금 만들고 있는 글자 목록입니다.
예: ["a", "j"]. 붙였다가 지울 수 있습니다.

answers 완성한 문자열을 모아 두는 답 목록입니다.
예: ["aj"]. 찾은 답은 여기에 계속 남깁니다.

차이 **path에서 j를 지워도 answers의 "aj"는 남습니다.**
다음 답 "ak"를 만들 때 path만 다시 사용합니다.

i는 다음에 처리할 숫자의 위치입니다

입력 위치

digits = "25"
0번 위치의 숫자는 "2", 1번 위치의 숫자는 "5"입니다.

i = 0

아직 첫 숫자를 처리하지 않았습니다.
path = []에서 a·b·c 중 하나를 고릅니다.

i = 1

첫 글자를 골랐습니다. 예: path = ["a"]
이제 둘째 숫자 "5"의 j·k·l 중 하나를 고릅니다.

i = 2

두 자리를 모두 채웠습니다. 예: ["a", "j"]
입력 길이 2와 같으므로 "aj"를 답으로 저장합니다.

숫자에 맞는 문자를 읽는 Python 사전

```python
keys = {"2": "abc", "5": "jkl", "9": "wxyz"}
keys["2"]    # "abc"
keys["5"]    # "jkl"

for ch in keys["5"]:
    print(ch)  # j, k, l을 하나씩 출력
```

키 "5"로 문자 묶음 "jkl"을 찾습니다.

그림의 움직임을 세 줄로 적으면

```
path.append(ch)
visit(i + 1)
path.pop()
```

문자 붙이기 → 다음 자리 처리 → 붙인 문자 지우기

visit가 끝나면 그다음 줄로 돌아옵니다

```
path.append(ch)
visit(i + 1)
path.pop()
```

지금 i=1, ch="j", path=["a"]라고 해봅니다.

첫 줄: j를 붙여 ["a","j"]가 됩니다.

둘째 줄: visit(2)가 "aj"를 저장합니다.

visit(2)가 끝나면 셋째 줄을 실행해 j를 지웁니다.

호출한 줄 아래로 돌아오기 때문에, pop은 다음 후보를 고르기 전에 실행됩니다.

입력 끝에 도착하면 답을 저장합니다

```
1    if i == len(digits):
2        answers.append("".join(path))
3        return
```

["a", "j"]를 문자열 "aj"로 합쳐 저장

전화 문자 함수의 입력과 반환

입력

digits: 2~9로 이루어진 문자열입니다.
빈 문자열이면 빈 목록 []을 반환합니다.

문자 연결

2: abc 3: def 4: ghi 5: jkl
6: mno 7: pqrs 8: tuv 9: wxyz

규칙

각 숫자에서 한 글자씩 골라 입력 순서대로 잇습니다.
완성한 문자열의 길이는 입력의 길이와 같습니다.

반환

이 규칙으로 만들 수 있는 모든 문자열의 목록.
예를 들어 입력 "2"의 답은 ["a", "b", "c"]입니다.

```
letter_combinations(digits)
```

def는 함수를 만들고, 괄호를 붙여 호출합니다

함수 정의

def visit(i): 아래의 들여쓴 줄은
visit가 해야 할 일을 적어 둔 것입니다.

실행 시작

visit(0)을 실행해야 첫 자리의 탐색이 시작됩니다.
함수를 정의한 것만으로 탐색이 시작되지는 않습니다.

재귀 호출

visit 안에서 visit(i+1)을 호출합니다.
같은 일을 다음 자리에 대해 하도록 맡기는 것입니다.

끝나는 조건

i가 입력 길이와 같으면 답을 저장하고 return합니다.
이 조건이 있으므로 계속 호출하지 않고 돌아옵니다.

전화 키패드 · Python 전체 코드

```python
def letter_combinations(digits):
    keys = {"2":"abc", "3":"def", "4":"ghi",
            "5":"jkl", "6":"mno", "7":"pqrs",
            "8":"tuv", "9":"wxyz"}
    answers, path = [], []
    def visit(i):
        if i == len(digits):
            answers.append("".join(path))
            return
        for ch in keys[digits[i]]:
            path.append(ch)
            visit(i + 1)
            path.pop()
    if digits:
        visit(0)
    return answers
```

visit(0)에서 출발합니다. path는 모든 호출이 함께 사용합니다.

"25"를 넣었을 때 처음 저장되는 답

```
visit(0)
  a 선택 → visit(1)
    j 선택 → visit(2)
      "aj" 저장 후 return
```

visit(0): "2"의 후보 중 a를 고릅니다.

visit(1): "5"의 후보 중 j를 고릅니다.

visit(2): 입력 길이 2에 도착했습니다.

answers=["aj"]. 아직 전체 탐색은 끝나지 않았습니다.

들여쓰기는 이 설명에서 호출의 깊이를 보여줍니다. 위 네 줄은 실행 흐름을 적은 것입니다.

"aj"를 저장한 뒤 "ak"를 만드는 과정

```
path = ["a", "j"]
path.pop()          # ["a"]
path.append("k")    # ["a", "k"]
visit(2)            # "ak" 저장
```

돌아온 뒤 지우는 것은 마지막 글자 j 하나입니다.

첫 글자 a는 남아 있어 같은 첫 자리로 계속 탐색합니다.

j 다음 후보인 k를 붙입니다.

다시 끝에 도착하면 answers=["aj","ak"]가 됩니다.

j·k·l을 다 본 다음에야 a를 지우고, 첫 자리의 다음 후보 b로 갑니다.

세 자리도 같은 동작을 한 번 더 합니다

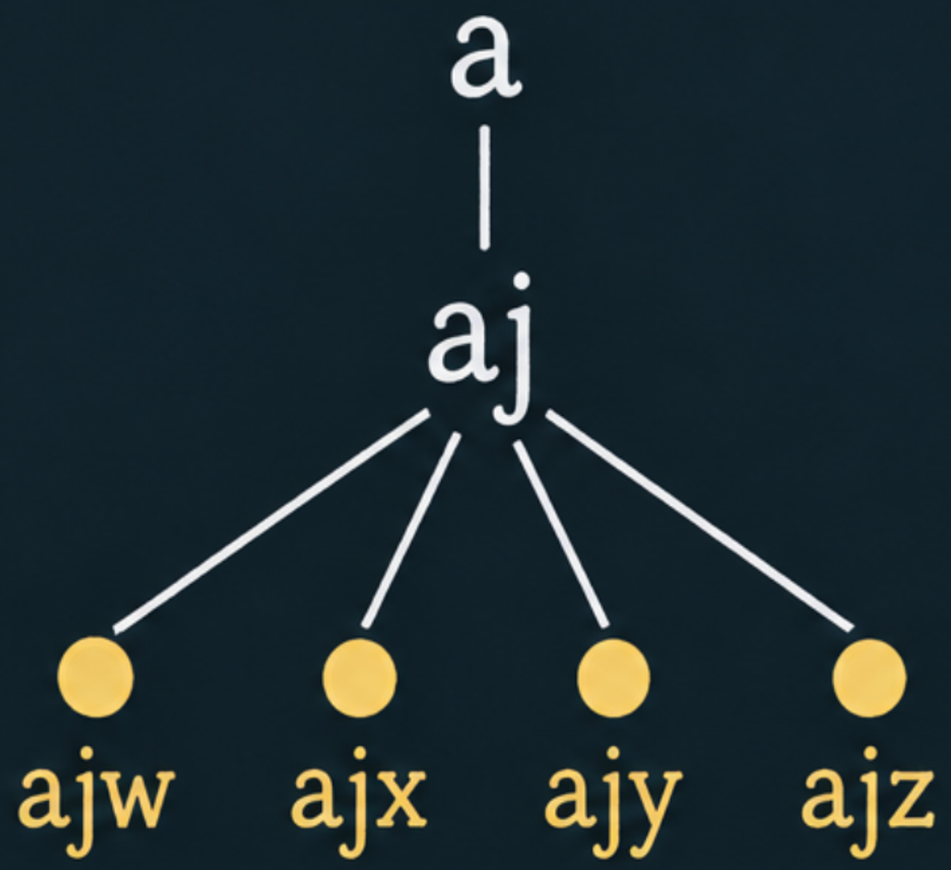
입력 "259"는 세 자리

2: a·b·c / 5: j·k·l

9: w·x·y·z

완성되는 문자열의 길이는 3

aj 뒤에서 마지막 문자만 바꿉니다



ajw를 저장한 뒤 w만 지웁니다.

ajx, ajy, ajz도 같은 방식입니다.

네 후보가 끝나면 j를 지웁니다.

그다음 ak에서 다시 w부터

답이 많으면 만드는 시간도 늘어납니다

25: 3 × 3 = 9개

259: 3 × 3 × 4 = 36개

n자리라면 답은 최대 4^n개

답 저장 시간 $O(n \cdot 4^n)$, 작업 공간 $O(n)$, 답 목록 공간은 별도

한 갈래를 본 뒤 다음 갈래를 보는 이유

a로 시작하는 답을 다 찾았으면 b로 시작하는 답도 봅니다.

이전 선택으로 돌아와 다음 후보를 골라야 답을 빠짐없이 찾습니다.

선택하기, 다음 단계 살펴보기, 선택 되돌리기

뒤의 문제에서는 조건을 만족하지 못한 갈래를 중간에 멈춥니다.

7-1 카드 배치 수 문제

상황

서로 다른 카드 n장에 1부터 n까지 적혀 있습니다.
그중 r장을 골라 왼쪽부터 한 줄로 놓습니다.

입력

n: 카드 수, 0~8의 정수
r: 놓을 카드 수, 0 ≤ r ≤ n

규칙

한 배치에서 같은 카드를 두 번 쓰지 않습니다.
놓인 순서가 다르면 다른 배치로 셉니다.

구할 답

가능한 배치의 수를 정수 하나로 반환합니다.
r=0이면 아무것도 놓지 않은 배치 1가지를 셉니다.

```
permutation_count(n, r)
```

7-1 · 서로 다른 카드로 두 자리를 채우면

카드 1·2·3에서 두 장을 놓습니다.

한 카드는 두 번 쓸 수 없습니다.

놓는 순서가 다르면 다른 배치입니다.

반환할 값은 배치의 개수

순서를 바꾸면 다른 배치가 됩니다

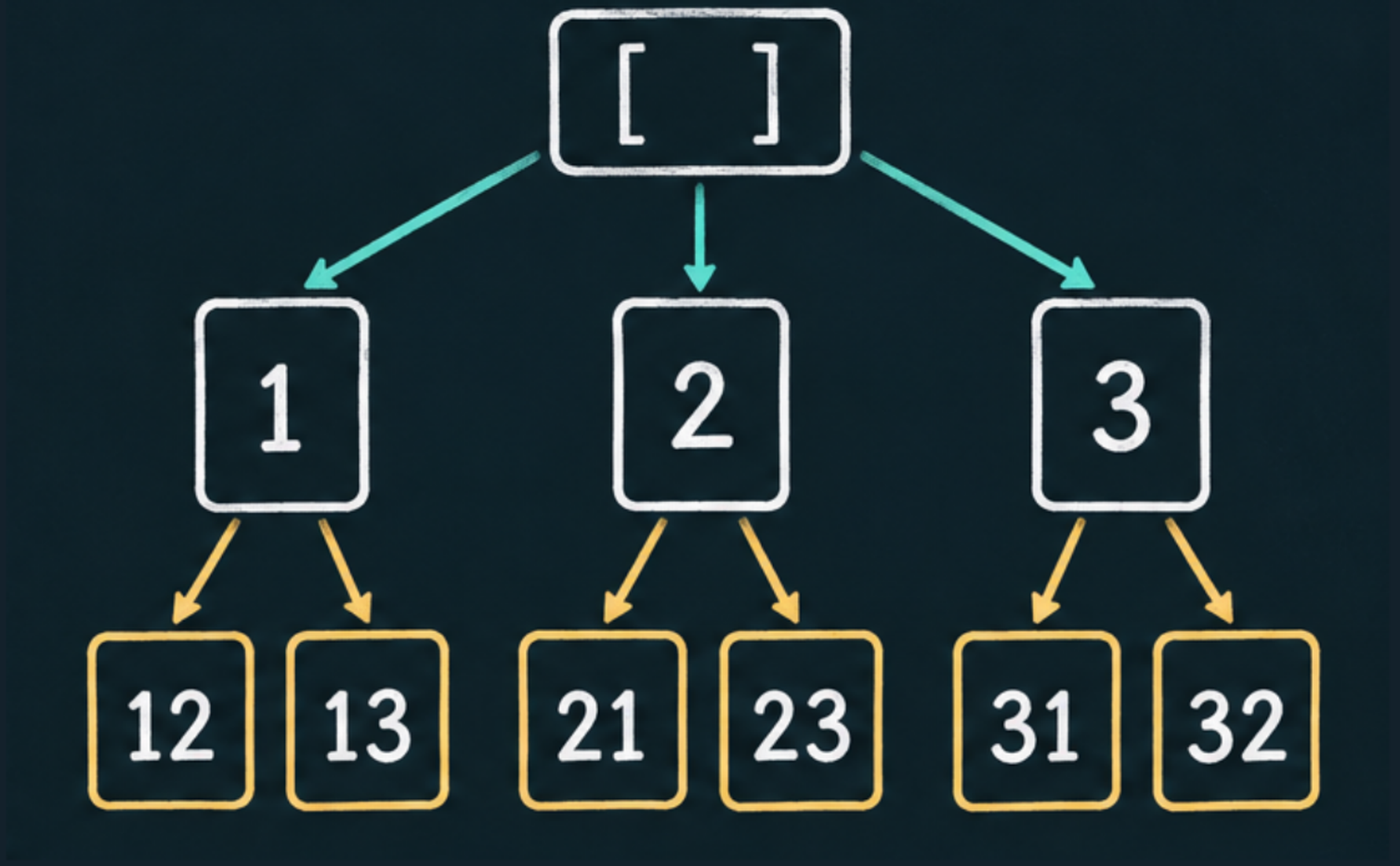
[1, 2]와 [2, 1]은 둘 다 셉니다.

[1, 1]은 같은 카드를 두 번 써서 불가

카드를 되돌려 놓고 다음 배치를 만듭니다.

다른 배치에서 다시 쓰는 것은 가능

끝까지 놓은 갈래가 여섯 개입니다



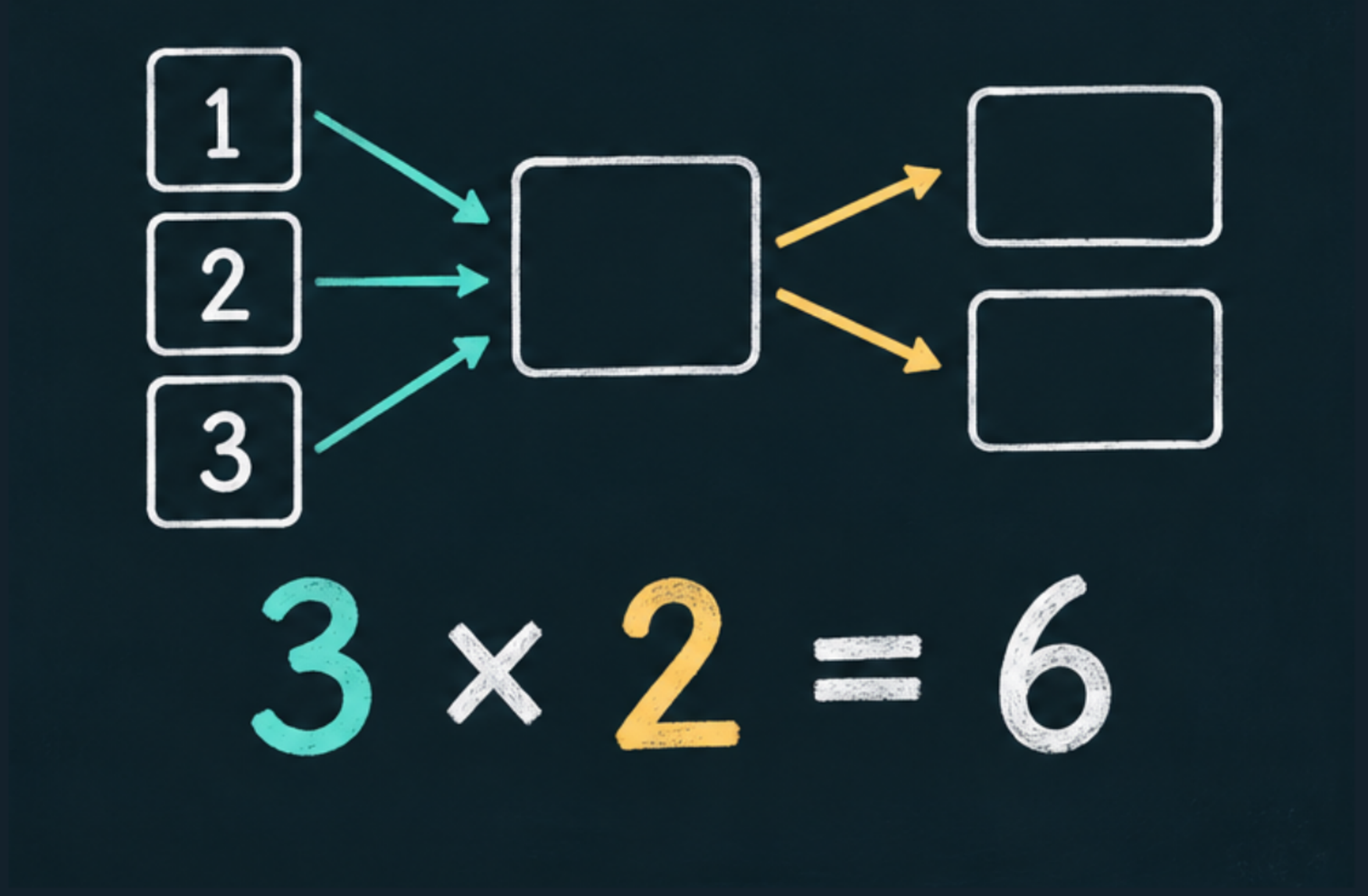
첫 카드마다 남은 두 장 중 선택

1 뒤에는 2 또는 3

2 뒤에는 1 또는 3, 3 뒤에는 1 또는 2

12, 13, 21, 23, 31, 32

개수만 필요하므로 선택 수를 곱합니다



첫 자리 3가지, 다음 자리 2가지

각 첫 선택에서 남은 카드 수가 같습니다.

3 × 2 = 6

n부터 r번, 하나씩 줄여 곱하기

7-1 · Python

```python
def permutation_count(n, r):
    answer = 1
    for i in range(r):
        answer *= n-i
    return answer
```

answer를 1로 시작하고, 각 자리의 선택 수를 곱합니다.

C의 for문은 세 부분으로 읽습니다

```
for (int i=0; i<r; ++i) {
    answer *= n-i;
}
```

int i=0: 처음에 i를 0으로 만듭니다.

i<r: 반복하기 전에 확인합니다. 거짓이면 끝납니다.

answer *= n-i: answer = answer * (n-i)입니다.

++i: 이번 반복이 끝난 뒤 i를 1 늘립니다.

예: r=2이면 i가 0, 1일 때 두 번 실행하고, i=2에서는 끝납니다.

7-1 · C

```
1   int permutation_count(int n, int r) {
2       int answer=1;
3       for (int i=0; i<r; ++i) answer*=n-i;
4       return answer;
5   }
```

int는 정수입니다. i++은 다음 자리로 한 번 이동합니다.

n=3, r=2를 변수 값으로 따라가기

시작: answer = 1

i=0: answer = 1 * (3-0) = 3

i=1: answer = 3 * (3-1) = 6

i=2: i<2가 거짓 → return 6

첫 자리는 카드 세 장 중 하나를 고릅니다.

두 번째 자리는 남은 카드 두 장 중 하나를 고릅니다.

세 번째 자리는 없으므로 두 번 곱한 뒤 끝냅니다.

r=0이면 한 번도 곱하지 않아 시작 값 1을 반환합니다.

처음 1은 선택할 자리가 없을 때의 빈 배치 한 가지이기도 합니다.

7-2 목표 합을 만드는 선택 수 문제

상황

배열 a에서 원소를 몇 개 골라 합을 만듭니다.
그 합이 target인 선택을 찾습니다.

입력

a: 길이 0~20, 각 값이 1~50인 양의 정수 배열
target: 목표 합, 0~1000의 정수

규칙

각 위치는 한 번만 고르고, 고른 순서는 구분하지 않습니다.
같은 값도 위치가 다르면 다른 원소입니다.

구할 답

합이 target인 선택의 수를 반환합니다.
아무것도 고르지 않은 선택의 합은 0으로 셉니다.

```
subset_sum_count(a, target)
```

7-2 · 합이 3이 되는 선택을 찾습니다

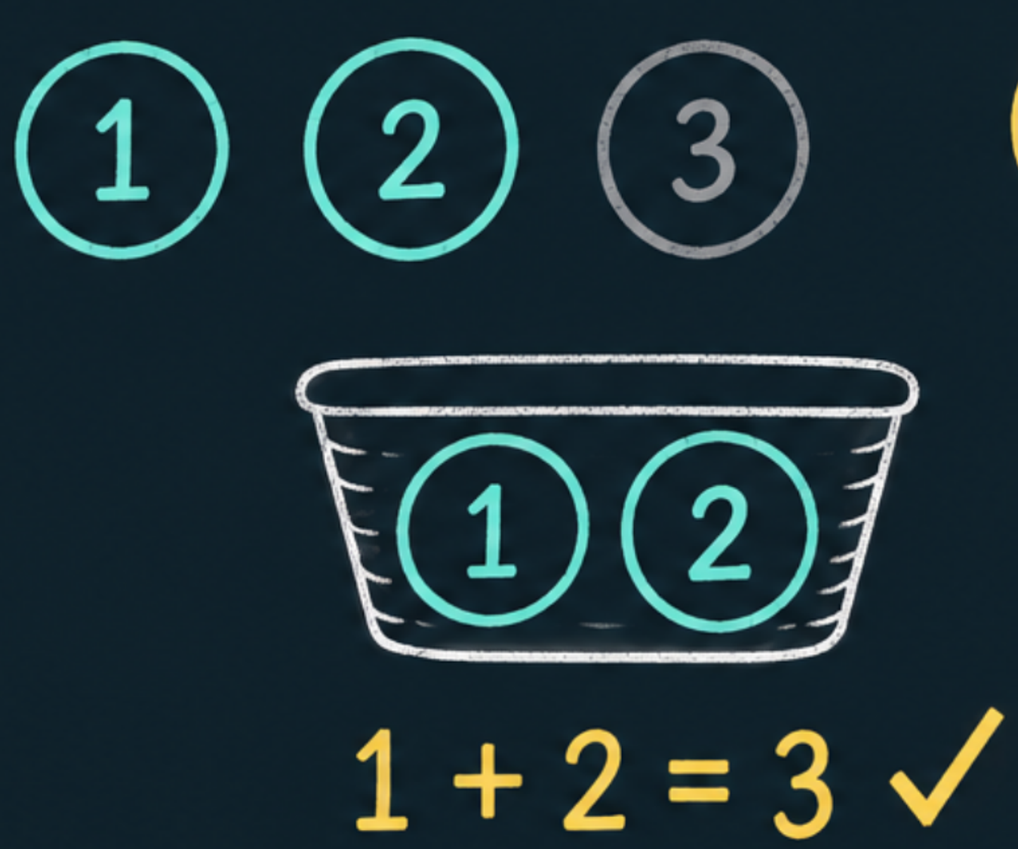
숫자 [1, 2, 3]에서 골라봅니다.

각 위치는 고르거나 건너뜁니다.

고른 순서는 구분하지 않습니다.

조건을 만족하는 선택의 수 반환

1과 2를 고르면 한 가지 성공



선택한 값의 합은 1 + 2 = 3

3은 고르지 않았습니다.

성공한 선택 하나로 셉니다.

현재 성공 수 1

3만 고르는 것도 다른 성공

1과 2는 건너뛰고 3을 고릅니다.

앞의 [1, 2]와 다른 선택입니다.

둘 다 목표 합 3을 만듭니다.

전체 답은 2

세 숫자에서는 여덟 선택을 확인합니다

```
선택 없음          합 0
[1]  [2]  [3]     합 1, 2, 3
[1,2] [1,3] [2,3] 합 3, 4, 5
[1,2,3]           합 6
```

각 숫자를 고르거나 건너뛰는 두 선택이 있습니다.

숫자가 세 개이므로 2×2×2=8개입니다.

목표가 3이면 [3], [1,2] 두 선택만 셉니다.

[2,1]을 따로 더하지 않습니다. 이미 같은 두 위치를 골랐습니다.

구할 것은 고른 숫자의 개수가 아니라, 합이 3인 서로 다른 선택의 개수입니다.

한 숫자를 볼 때는 고르기와 건너뛰기

search(i, total): i번부터 결정, 현재 합은 total

건너뛰기: 다음 위치, 같은 합

고르기: 다음 위치, total + a[i]

두 갈래에서 얻은 성공 수를 더합니다.

search(1, 1)을 우리말로 읽으면

고정 입력

a=[1,2,3], target=3입니다.
배열과 목표는 탐색 중에도 그대로 둡니다.

i = 1

0번 원소에 대한 결정은 끝났습니다.
이제 1번 원소인 숫자 2를 볼 차례입니다.

total = 1

앞에서 숫자 1을 골랐으므로
지금까지 고른 값의 합은 1입니다.

다음 선택

2를 건너뛰면 search(2,1),
2를 고르면 search(2,3)을 호출합니다.

한 갈래가 돌려주는 1은 성공 한 가지입니다

```
search(3, 3) → 1
search(3, 4) → 0

전체 성공 수:
건너뛴 쪽 + 고른 쪽
```

i=3이면 세 숫자의 선택을 모두 정했습니다.

합 3인 완성 선택이면 1을 돌려줍니다.

합이 다르면 이번 선택은 실패라 0을 돌려줍니다.

위의 호출들이 이 1과 0을 더해 전체 성공 수를 모읍니다.

0과 1은 이 문제에서 실패·성공한 선택의 개수이며, 마지막에는 합계 2가 나옵니다.

이미 목표를 넘은 갈래는 멈춥니다

예: [2, 4, 1], 목표 3에서 2와 4를 선택

현재 합 6에 남은 양수를 더해도 3이 될 수 없음

6 > 3이면 이 갈래의 성공 수는 0

이 문제의 모든 원소가 양수라는 조건을 사용합니다.

아무것도 고르지 않는 것도 선택 한 가지

선택 방법
모든 원소를 건너뛰는 방법이 있습니다.
이 방법에서 고른 원소는 없습니다.

합
고른 값이 없으므로 합은 0입니다.
문제에서 이 빈 선택도 포함하기로 했습니다.

target = 0
양수는 하나라도 고르면 합이 커집니다.
따라서 빈 선택 한 가지가 답이고 1을 반환합니다.

빈 배열
a=[]이면 고를 원소 자체가 없습니다.
target=0이면 1, target가 양수이면 0입니다.

7-2 · Python

```python
def subset_sum_count(a, target):
    def search(i, total):
        if total > target:
            return 0
        if i == len(a):
            return int(total == target)
        return (search(i+1, total)
                + search(i+1, total+a[i]))
    return search(0, 0)
```

끝까지 결정한 뒤 목표와 같으면 1, 다르면 0을 반환합니다.

C에서는 두 함수를 나란히 작성합니다

위쪽 함수 subset_search는 실제 탐색을 하는 보조 함수입니다.
입력 배열·목표·현재 위치·현재 합을 받습니다.

아래쪽 함수 subset_sum_count는 사이트가 처음 호출하는 함수입니다.
현재 위치 0, 현재 합 0으로 탐색을 시작합니다.

작성 위치 두 함수는 서로의 중괄호 밖에 나란히 씁니다.
C에서는 함수 안에 다른 함수를 정의하지 않습니다.

배열 읽기 **a는 배열, n은 원소 수입니다. const는 읽기 전용 표시입니다.**
static은 이 파일 안에서 쓸 보조 함수라는 표시입니다.

7-2 · C

```c
static int subset_search(const int a[], int n, int target,
                         int i, int total) {
    if (total>target) return 0;
    if (i==n) return total==target;
    return subset_search(a,n,target,i+1,total)
         + subset_search(a,n,target,i+1,total+a[i]);
}
int subset_sum_count(const int a[], int n, int target) {
    return subset_search(a,n,target,0,0);
}
```

subset_search는 탐색용 함수, subset_sum_count는 시작 함수입니다.

7-3 이웃한 점을 다른 색으로 칠하는 문제

상황

선으로 연결된 점끼리 색이 다르도록
모든 점에 색 하나씩을 칠합니다.

입력

graph: 점 1~8개의 연결을 0과 1로 나타낸 표
k: 사용할 수 있는 색 수, 1~3의 정수

규칙

연결되지 않은 점은 같은 색이어도 됩니다.
주어진 색을 전부 사용할 의무는 없습니다.

구할 답

모든 점을 칠할 방법이 하나라도 있으면 1, 없으면 0.
떨어져 있는 점도 모두 칠해야 합니다.

```
can_color(graph, k)
```

graph는 점 사이의 연결을 적은 표

```
     0   1   2
0  [0,  1,  0]
1  [1,  0,  1]
2  [0,  1,  0]
```

점 번호는 0, 1, 2입니다.

graph[0][1] = 1
0번과 1번 사이에 선이 있습니다.

graph[0][2] = 0
0번과 2번 사이에는 선이 없습니다.

표는 좌우 대칭이며, 자기 자신과의 연결을 나타내는 대각선은 0입니다.

7-3 · 선으로 연결된 점은 다른 색

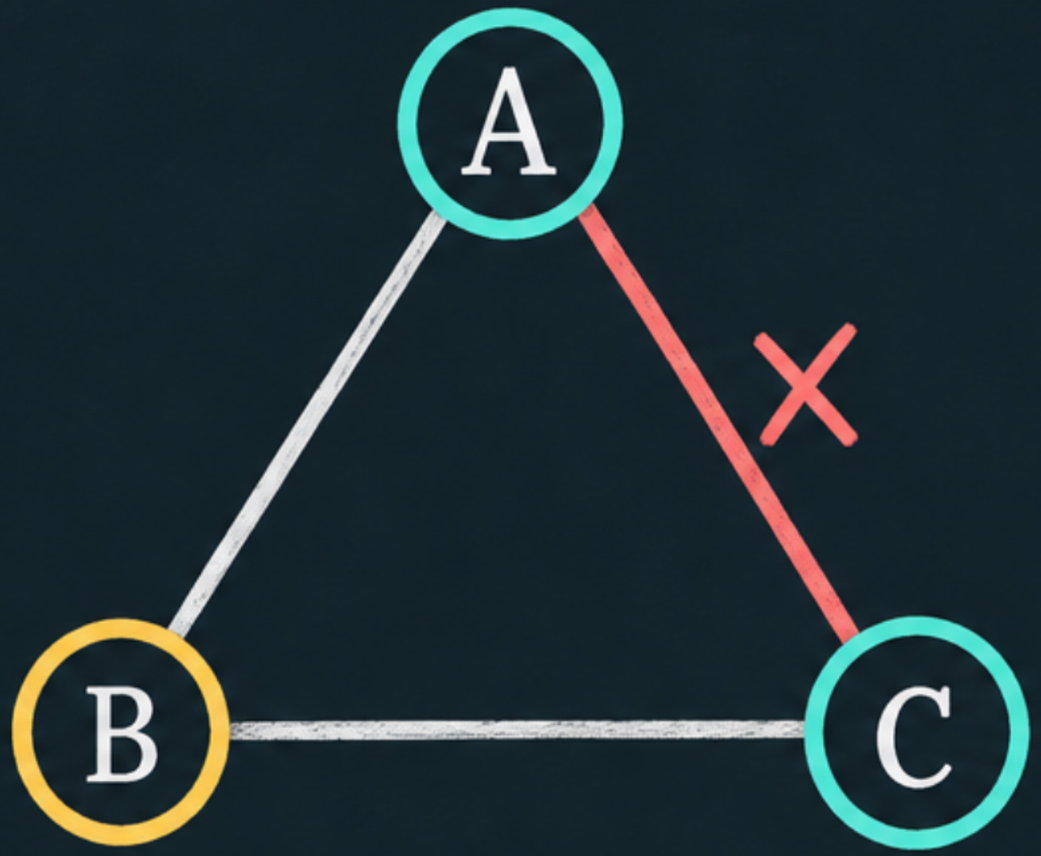
A와 C는 연결되지 않았습니다.

A와 C가 같은 색이어도 됩니다.

A–B, B–C의 양 끝 색만 다르면 됩니다.

가능하면 1, 불가능하면 0

삼각형에는 두 색으로 부족합니다



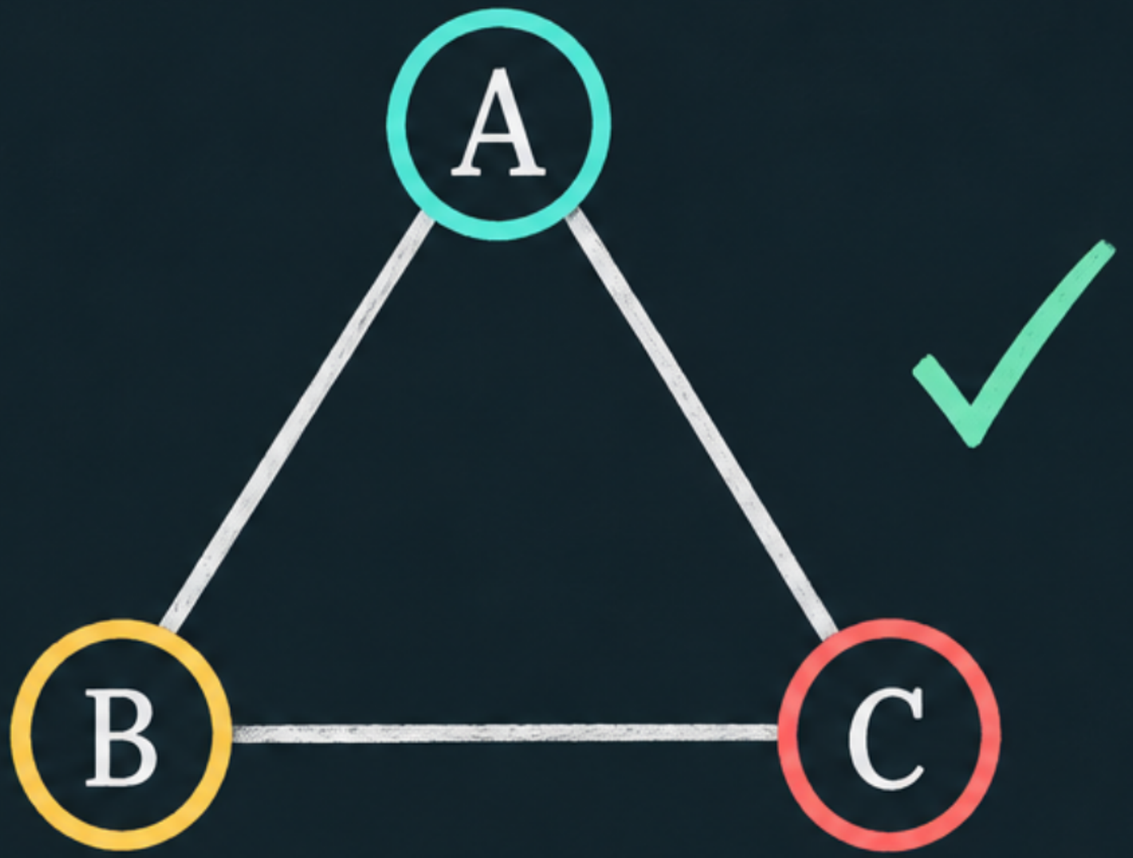
A는 초록, B는 노랑으로 칠했습니다.

C를 초록으로 칠하면 A와 충돌

C를 노랑으로 칠하면 B와 충돌

두 색으로는 완성할 수 없음

세 번째 색이 있으면 완성됩니다



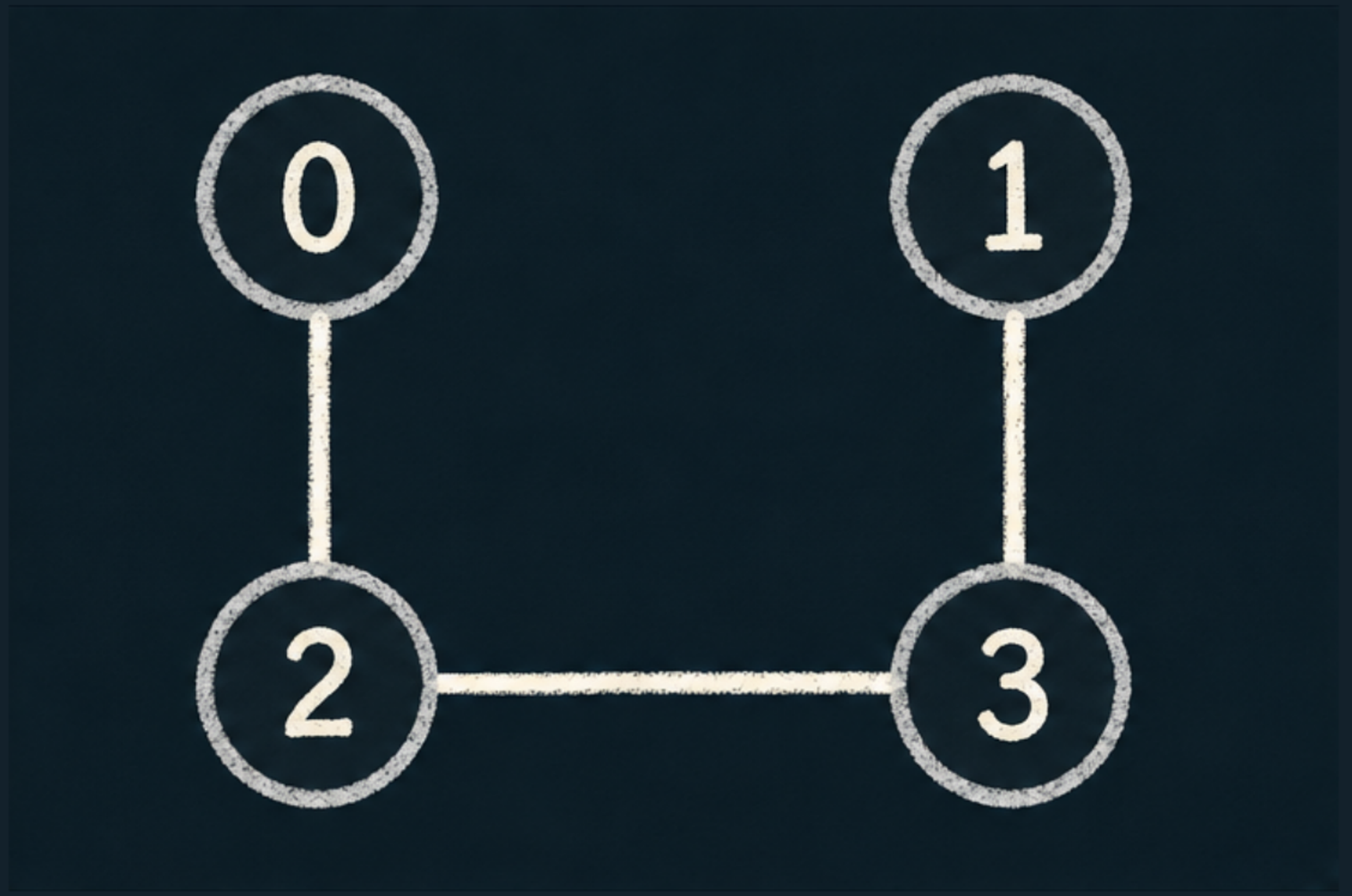
A·B·C가 모두 다른 색입니다.

세 선의 양 끝 색이 모두 다릅니다.

사용 가능한 색을 전부 쓸 의무는 없습니다.

k = 3이면 1 반환

네 점을 두 색으로 칠해 보겠습니다



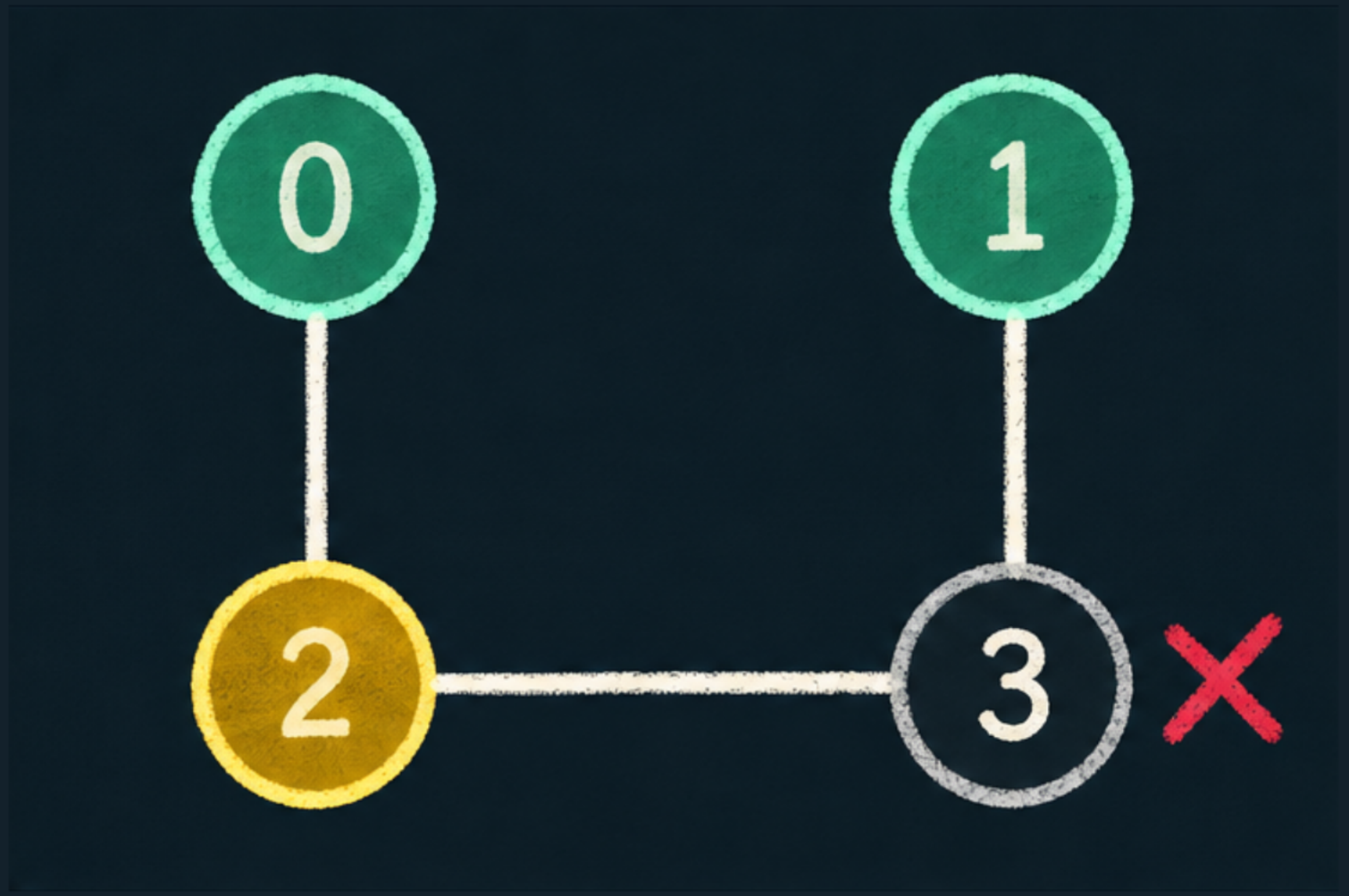
점은 0, 1, 2, 3입니다.

선은 0–2, 2–3,
3–1뿐입니다.

0번부터 번호 순서대로
칠합니다.

**초록과 노랑, 두 색만
사용**

지금 허용된 색을 골라도 나중에 막힙니다



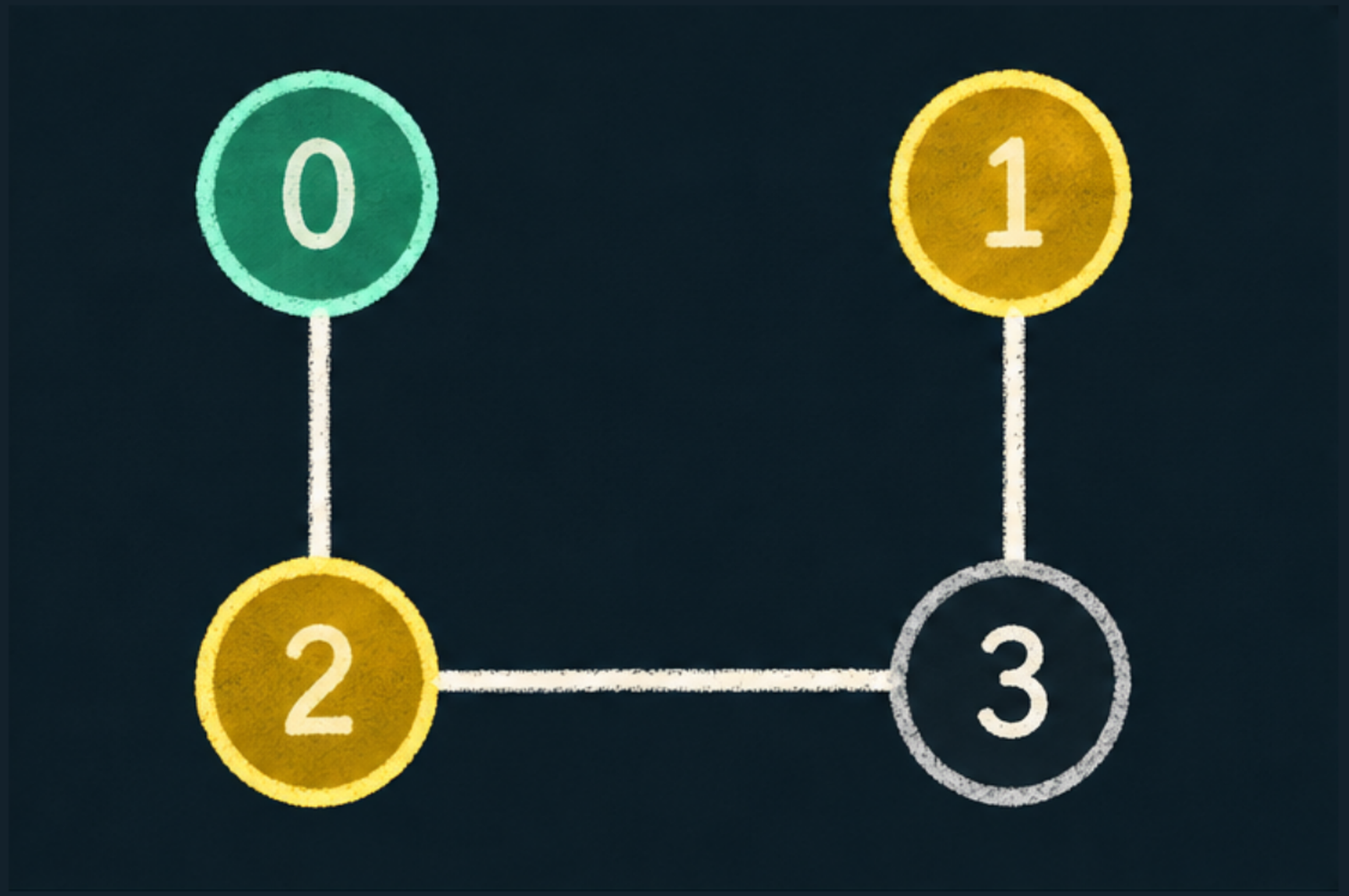
0번 초록, 1번 초록, 2번 노랑

1번은 0번과 연결되지 않아 초록 가능

3번은 초록·노랑 이웃을 둘 다 만납니다.

3번에서 쓸 수 있는 색이 없음

1번의 색을 바꾸고 다시 진행합니다



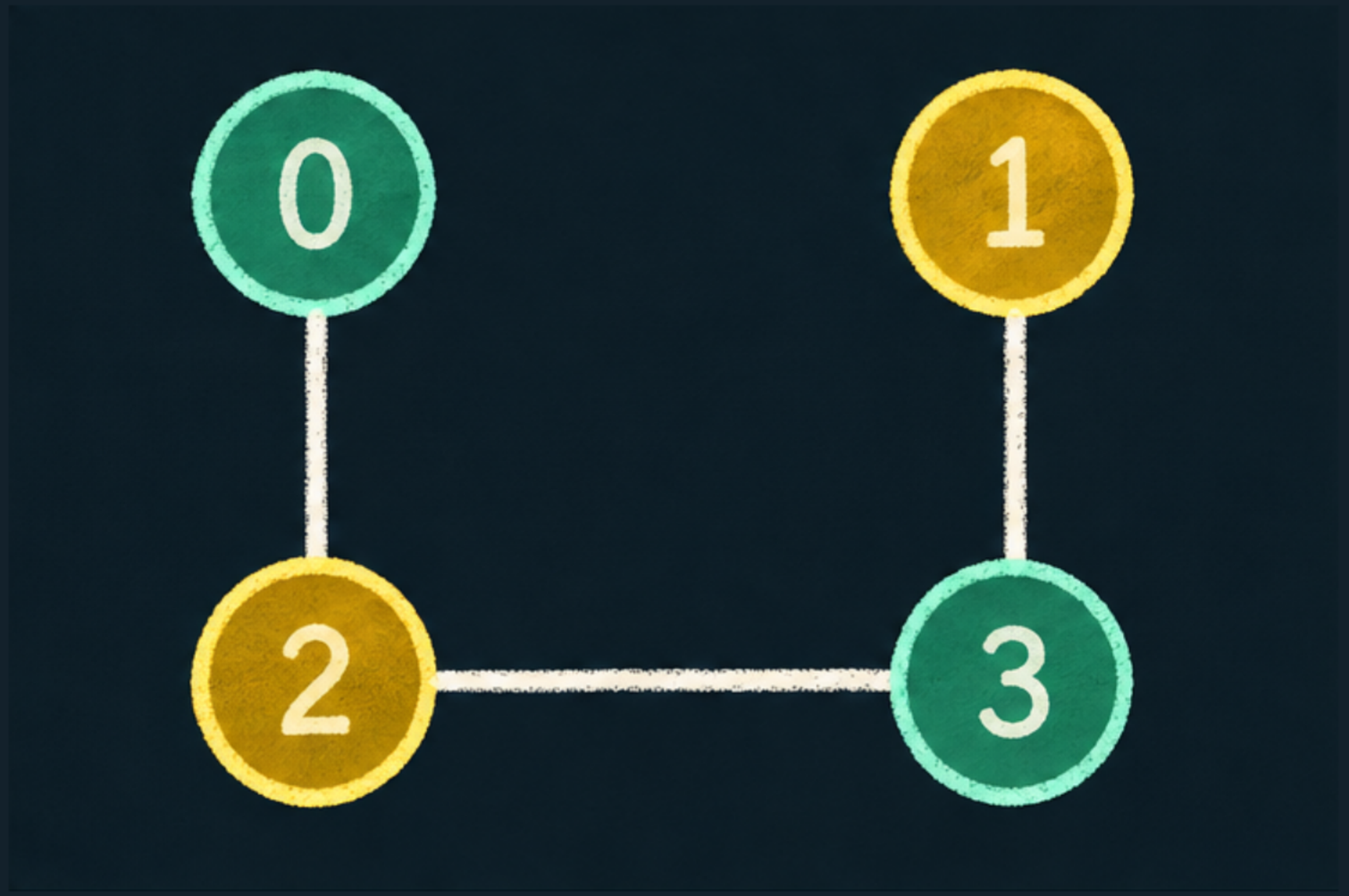
1번의 초록을 취소하고 노랑으로

0번은 초록으로 그대로 둡니다.

2번은 0번과 다르게 노랑을 쓸 수 있습니다.

이제 3번에서 초록을 고를 수 있음

네 점을 모두 칠했으므로 1을 반환합니다



색 배열은 [초록, 노랑, 노랑, 초록]

0–2, 2–3, 3–1의 양 끝이 다릅니다.

모든 점을 칠한 방법 하나를 찾았습니다.

가능하므로 함수의 답은 1

color 배열에는 지금까지 정한 색을 적습니다

숫자로 저장 0은 아직 안 칠함, 1은 초록, 2는 노랑으로 정합니다.
처음 color=[0,0,0,0]입니다.

한 번 선택 0번을 초록으로 칠하면 color[0]=1입니다.
color=[1,0,0,0]이 됩니다.

되돌리기 1번에 정한 색이 실패하면 color[1]=0으로 지웁니다.
그 자리에 다음 후보 색을 넣어 봅니다.

다음 점 **paint(v)는 v번부터 칠하는 함수입니다.**
모든 점을 칠해 v=n이 되면 성공 1을 반환합니다.

7-3 · Python (1/2)

```python
def can_color(graph, k):
    n = len(graph)
    color = [0]*n
    def paint(v):
        if v == n:
            return 1
        for c in range(1, k+1):
            allowed = True
            for u in range(v):
                if graph[v][u] == 1 and color[u] == c:
                    allowed = False
                    break
```

다음 화면에서 이어집니다.

7-3 · Python (2/2)

```
            if allowed:
                color[v] = c
                if paint(v+1) == 1:
                    return 1
                color[v] = 0
        return 0
    return paint(0)
```

앞 화면과 이어지는 같은 함수입니다.

7-3 · C (1/2)

```
1   static int paint(const int g[][8], int n, int k,
2                    int v, int color[]) {
3       if (v==n) return 1;
4       for (int c=1; c<=k; ++c) {
5           int ok=1;
6           for (int u=0; u<v; ++u)
7               if (g[v][u] && color[u]==c) ok=0;
```

다음 화면에서 이어집니다.

7-3 · C (2/2)

```
8             if (ok) { color[v]=c;
9                 if (paint(g,n,k,v+1,color)) return 1;
10                color[v]=0; }
11        } return 0;
12    }
13    int can_color(const int graph[][8], int n, int k) {
14        int color[8]={0}; return paint(graph,n,k,0,color);
15    }
```

앞 화면과 이어지는 같은 함수입니다.

7-4 서로 공격하지 않는 퀸 배치 수 문제

상황

n×n 체스판에 퀸 n개를 놓습니다.
어느 두 퀸도 서로 공격하지 않아야 합니다.

입력

n: 체스판 한 변의 칸 수이자 놓을 퀸 수
1~8의 정수입니다.

규칙

같은 행·같은 열·같은 대각선에 두 퀸을 놓지 않습니다.
회전·대칭이어도 놓인 칸이 다르면 다른 배치입니다.

구할 답

퀸 n개를 모두 놓을 수 있는 배치의 수입니다.
가능한 배치가 없으면 0을 반환합니다.

```
nqueen_count(n)
```

7-4 · 퀸은 가로·세로·대각선으로 공격

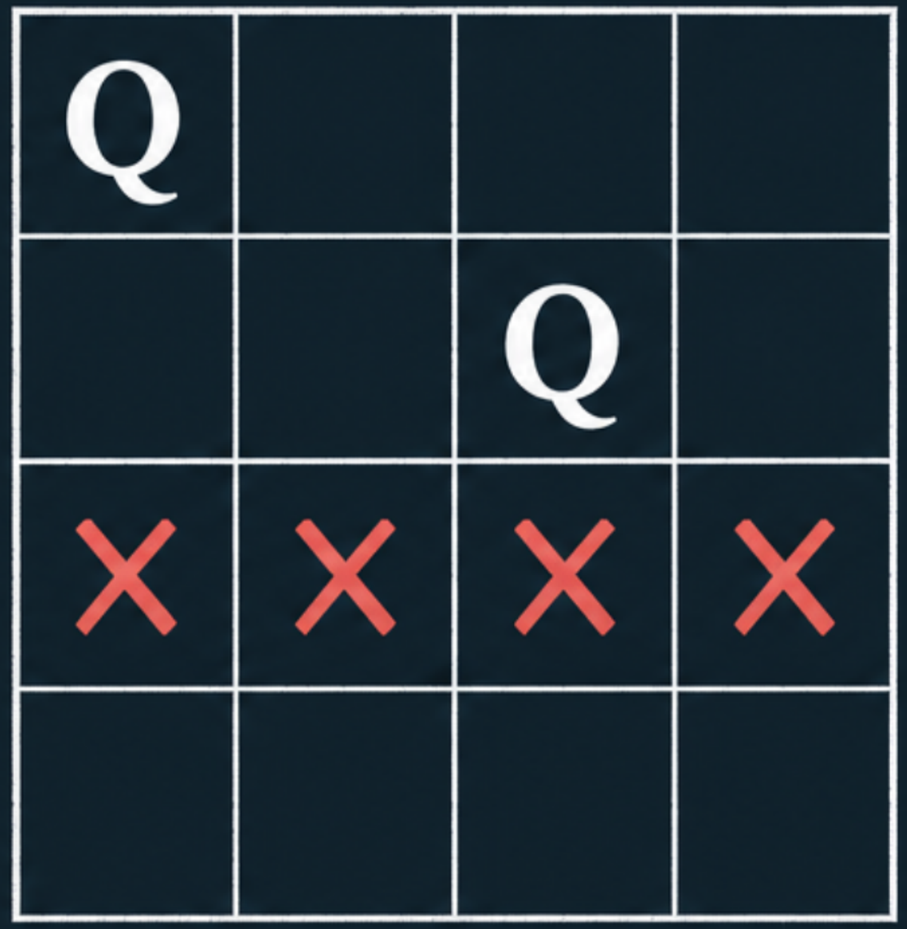
색칠된 칸은 같은 퀸이 공격합니다.

n×n 판에 퀸 n개를 놓습니다.

두 퀸이 서로 공격하면 안 됩니다.

가능한 배치의 개수 반환

첫 선택 때문에 나중 행이 막힐 수 있습니다



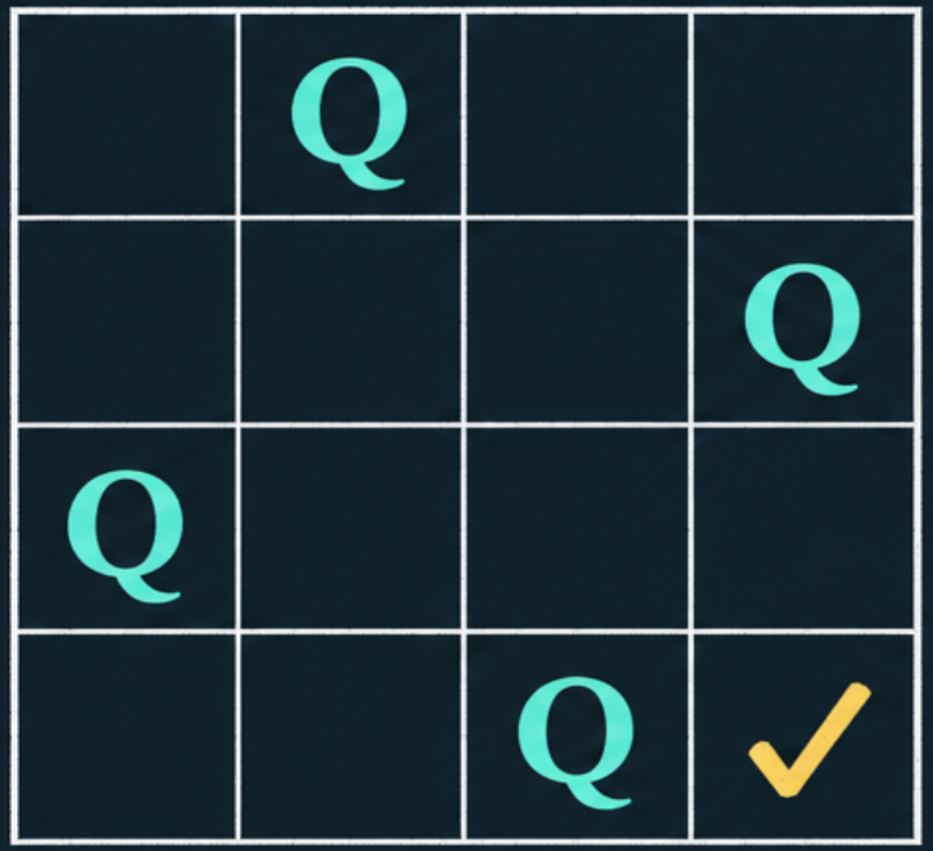
1행 1열, 2행 3열에 놓아봅니다.

3행에는 안전한 칸이 없습니다.

직전 행의 퀸을 빼고 다른 열을 봅니다.

막히면 이전 선택으로 돌아가기

첫 행의 열을 바꾸면 끝까지 놓을 수 있습니다



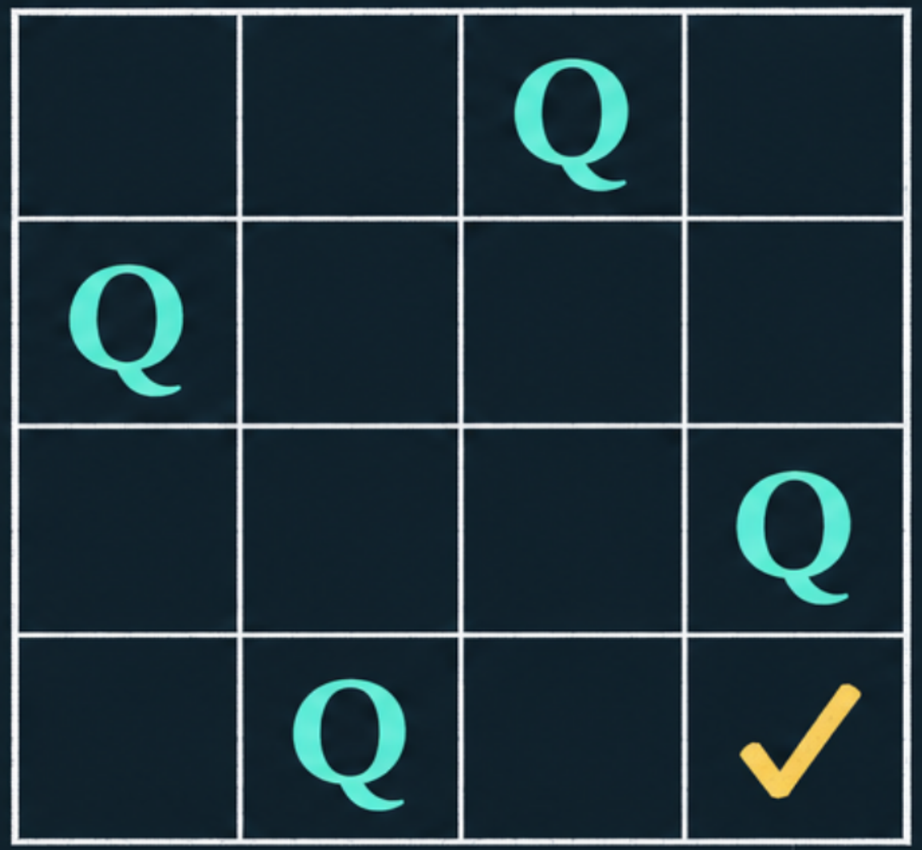
위에서부터 2·4·1·3열

같은 열에 두 퀸이 없습니다.

대각선에도 두 퀸이 겹치지 않습니다.

성공 배치 한 가지

좌우를 뒤집은 배치도 따로 셉니다



위에서부터 3·1·4·2열

앞의 2·4·1·3열과 다른 배치입니다.

4×4 판에서는 두 배치가 전부입니다.

n = 4의 답은 2

대각선 검사를 숫자로 따라가기

같은 열 앞 퀸이 0번 행 1번 열에 있다고 합시다.
새 퀸을 2번 행 1번 열에 놓으면 열이 같아 안 됩니다.

대각선 충돌 새 퀸을 1번 행 2번 열에 놓아 봅니다.
행 차이 1, 열 차이 1이므로 같은 대각선입니다.

반대 대각선 1번 행 0번 열도 행 차이 1, 열 차이 -1입니다.
열 차이의 절댓값을 쓰면 이쪽도 1로 잡힙니다.

안전한 예 **1번 행 3번 열은 행 차이 1, 열 차이 2입니다.**
열도 다르고 대각선도 달라 이 앞 퀸과 충돌하지 않습니다.

columns에는 각 행의 퀸 위치를 적습니다

번호 기준 코드에서 행과 열은 0부터 셉니다.
그림의 1번째 열은 코드의 0번 열입니다.

예: [1,3] 0번 행의 퀸은 1번 열에 있습니다.
1번 행의 퀸은 3번 열에 있습니다.

다음 행 앞의 두 행을 채웠으므로 row=2입니다.
2번 행에서 안전한 열을 고릅니다.

놓고 빼기 **0번 열에 놓으면 append(0)으로 [1,3,0]입니다.**
돌아오면 pop()으로 다시 [1,3]을 만듭니다.

7-4 · Python (1/2)

```python
def nqueen_count(n):
    columns = []
    def place(row):
        if row == n:
            return 1
        answer = 0
        for col in range(n):
            allowed = True
            for r in range(row):
                c = columns[r]
                if col == c or abs(col-c) == row-r:
                    allowed = False
                    break
```

다음 화면에서 이어집니다.

7-4 · Python (2/2)

```
14              if allowed:
15                  columns.append(col)
16                  answer += place(row+1)
17                  columns.pop()
18          return answer
19      return place(0)
```

앞 화면과 이어지는 같은 함수입니다.

7-4 · C

```
1   static int queens(int n, int row, int pos[]) {
2       if (row==n) return 1;
3       int answer=0;
4       for (int col=0; col<n; ++col) {
5           int ok=1;
6           for (int r=0; r<row; ++r) {
7               int d=col-pos[r]; if (d<0) d=-d;
8               if (col==pos[r] || d==row-r) ok=0;
9           }
10          if (ok) {pos[row]=col; answer+=queens(n,row+1,pos);}
11      } return answer;
12  }
13  int nqueen_count(int n) {
14      int pos[8]={0}; return queens(n,0,pos);
15  }
```

pos[row]는 row행에 놓은 퀸의 열입니다.

문제마다 무엇을 반환하는지 먼저 확인합니다

전화 문자: 완성된 문자열 목록

카드·목표 합·퀸: 성공한 경우의 수

색칠: 가능하면 1, 불가능하면 0

성공한 즉시 끝낼지, 모든 성공을 더할지도 달라집니다.

선택과 되돌리기를 직접 따라가며 구현

7-1 카드 배치 수 · 7-2 목표 합 선택 수

7-3 그래프 색칠 가능 여부 · 7-4 퀸 배치 수

지금 선택한 것과 다음 위치를 적기

작은 예를 손으로 풀고, 같은 결과가 나오는지 실행합니다.